CIPHER 9.0 CP Set 1

Problem 1: The Detective's Longest Clue

Statement

A detective discovers a mysterious string hidden inside an old document. The detective believes that the most important clues are continuous portions of the string that read identically from both directions.

Your task is to identify all such distinct longest substrings present in the given string.

A single character is also considered a valid palindrome.

Input Format

A single line containing a string S.

Output Format

Print the following:
1. The maximum length of a palindromic substring.
2. On each subsequent line, print one distinct longest palindromic substring, ordered by its first occurrence in S.

Constraints

- $1 \leq |S| \leq 2000$
- S contains only lowercase English letters.

Sample Input

```
babad
```

Sample Output

```
3
bab
aba
```

Explanation

Both “bab” and “aba” are palindromic substrings of length 3. No longer palindromic substring exists. They are printed in order of first appearance.

10 Additional Test Cases

Test Case 1

Input

```
a
```

Output

```
1
a
```

Test Case 2

Input

```
aaaa
```

Output

```
4
aaaa
```

Test Case 3

Input

```
abc
```

Output

```
1
a
b
c
```

Test Case 4

Input

```
abacdfgdcaba
```

Output

```
3
aba
```

Test Case 5

Input

```
cbbd
```

Output

```
2
bb
```

Test Case 6

Input

```
forgeeksskeegfor
```

Output

```
10
geeksskeeg
```

Test Case 7

Input

```
abacaba
```

Output

```
7
abacaba
```

Test Case 8

Input

```
noonabbad
```

Output

```
4
noon
abba
```

Test Case 9

Input

```
racecarxyz
```

Output

```
7
racecar
```

Test Case 10

Input

```
abcddcbaef
```

Output

```
8
abcddcba
```

Problem 2: Circular Evacuation

Statement

A city has N people standing in a circle, numbered 1 to N in clockwise order. An evacuation officer starts counting from person 1.

The officer counts K people clockwise, including the person from whom the count begins. The person on whom the count ends is evacuated. Counting then resumes from the next person clockwise, and the process continues until only one person remains.

Determine the complete order in which people are evacuated and identify the final survivor.

Input Format

A single line containing two space-separated integers N and K.

Output Format

Print two lines:
Elimination: followed by the N−1 eliminated person numbers in order.
Survivor: followed by the number of the last remaining person.

Constraints

- 1 ≤ N ≤ 100000
- 1 ≤ K ≤ 10^9

Sample Input

```
7 3
```

Sample Output

```
Elimination: 3 6 2 7 5 1
Survivor: 4
```

Explanation

Starting from person 1, counting three people eliminates person 3. Counting resumes at person 4, so person 6 is eliminated next. Continuing in the same way gives the order 3, 6, 2, 7, 5, 1. Person 4 remains.

10 Additional Test Cases

Test Case 1

Input

```
1 1
```

Output

```
Elimination: NONE
Survivor: 1
```

Test Case 2

Input

```
2 1
```

Output

```
Elimination: 1
Survivor: 2
```

Test Case 3

Input

```
2 2
```

Output

```
Elimination: 2
Survivor: 1
```

Test Case 4

Input

```
5 1
```

Output

```
Elimination: 1 2 3 4
Survivor: 5
```

Test Case 5

Input

```
5 2
```

Output

```
Elimination: 2 4 1 5
Survivor: 3
```

Test Case 6

Input

```
5 3
```

Output

```
Elimination: 3 1 5 2
Survivor: 4
```

Test Case 7

Input

```
6 4
```

Output

```
Elimination: 4 2 1 3 6
Survivor: 5
```

Test Case 8

Input

```
8 2
```

Output

```
Elimination: 2 4 6 8 3 7 5
Survivor: 1
```

Test Case 9

Input

```
10 3
```

Output

```
Elimination: 3 6 9 2 7 1 8 5 10
Survivor: 4
```

Test Case 10

Input

```
10 7
```

Output

```
Elimination: 7 4 2 1 3 6 10 5 8
Survivor: 9
```

Problem 3: The Cursed Text Message

Statement

A communication device has a strange typing mechanism. Whenever two consecutive identical characters appear in a message, both characters disappear immediately.

After a pair disappears, the characters that were on either side become adjacent. If those characters are also identical, they disappear as well. This process continues until no two adjacent characters are identical.

Given the original message, determine the final message after all possible cancellations.

Input Format

A single line containing a string S.

Output Format

Print the final string. If every character disappears, print EMPTY.

Constraints

- $1 \le |S| \le 200000$
- S contains only lowercase English letters.

Sample Input

```
abbaca
```

Sample Output

```
ca
```

Explanation

The adjacent pair “bb” disappears first, leaving “aaca”. The newly adjacent pair “aa” then disappears, leaving “ca”. No further cancellation is possible.

10 Additional Test Cases

Test Case 1

Input

```
a
```

Output

```
a
```

Test Case 2

Input

```
aa
```

Output

```
EMPTY
```

Test Case 3

Input

```
ab
```

Output

```
ab
```

Test Case 4

Input

```
aabb
```

Output

```
EMPTY
```

Test Case 5

Input

```
azxxzy
```

Output

```
ay
```

Test Case 6

Input

```
abba
```

Output

```
EMPTY
```

Test Case 7

Input

```
aaaa
```

Output

```
EMPTY
```

Test Case 8

Input

```
abcddcba
```

Output

```
EMPTY
```

Test Case 9

Input

```
abbccdde
```

Output

```
ae
```

Test Case 10

Input

```
abccbaab
```

Output

```
ab
```

Problem 4: The Editor That Remembers Too Much

Statement

A cryptographer is writing a message in a simple text editor. The editor supports typing, cursor movement, deletion, undo and redo operations.

The cursor is represented by the number of characters to its left. For example, in the message “hello”, a cursor position of 2 means the cursor is between “e” and “l”.

The editor supports the following commands:
• TYPE c — Insert the lowercase character c at the current cursor position. The cursor moves one position to the right.
• LEFT — Move the cursor one position to the left, if possible.
• RIGHT — Move the cursor one position to the right, if possible.
• BACKSPACE — Delete the character immediately to the left of the cursor, if one exists. The cursor moves one position to the left.
• UNDO — Undo the most recent effective TYPE or BACKSPACE operation that has not already been undone.
• REDO — Reapply the most recently undone edit, if one exists.

Cursor movement commands do not themselves create undo history. However, undoing an edit restores the cursor position that existed immediately before that edit; redoing it restores the cursor position immediately after that edit.

An ineffective BACKSPACE, LEFT, or RIGHT command changes nothing and creates no history entry. An UNDO or REDO command with no available operation also changes nothing. Whenever a new effective edit is performed after an UNDO, the redo history is cleared.

After processing all commands, print the final message and the cursor position.

Input Format

The first line contains an integer Q, the number of commands.
Each of the next Q lines contains one command in one of these forms:
TYPE c
LEFT
RIGHT
BACKSPACE
UNDO
REDO

Output Format

Print two lines:
1. The final message, or EMPTY if the message is empty.
2. CURSOR p, where p is the number of characters to the left of the cursor (zero-based insertion position).

Constraints

- $1 \le Q \le 200000$
- The total number of commands is Q.
- The character c in TYPE c is a lowercase English letter.

Sample Input

```
8
TYPE a
TYPE b
LEFT
TYPE c
UNDO
UNDO
REDO
RIGHT
```

Sample Output

```
ab
CURSOR 2
```

Explanation

After typing “a” and “b”, the message is “ab” and the cursor is at position 2. LEFT moves the cursor to position 1. TYPE c inserts “c” before “b”, producing “acb”. The first UNDO removes “c” and restores the cursor to position 1. The second UNDO removes “b” and restores the cursor to position 1 (the cursor position before typing “b”). REDO restores “b” and its post-edit cursor position 2. RIGHT cannot move beyond the end of “ab”, so the final message is “ab” and the cursor is at position 2.

10 Additional Test Cases

Test Case 1

Input

```
1
TYPE a
```

Output

```
a
CURSOR 1
```

Test Case 2

Input

```
5
TYPE a
TYPE b
TYPE c
LEFT
BACKSPACE
```

Output

```
ac
CURSOR 1
```

Test Case 3

Input

```
3
TYPE a
TYPE b
UNDO
```

Output

```
a
CURSOR 1
```

Test Case 4

Input

```
4
TYPE a
TYPE b
UNDO
REDO
```

Output

```
ab
CURSOR 2
```

Test Case 5

Input

```
4
TYPE a
LEFT
BACKSPACE
RIGHT
```

Output

```
a
CURSOR 1
```

Test Case 6

Input

```
6
TYPE a
TYPE b
LEFT
TYPE x
UNDO
REDO
```

Output

```
axb
CURSOR 2
```

Test Case 7

Input

```
7
TYPE a
TYPE b
TYPE c
LEFT
LEFT
BACKSPACE
UNDO
```

Output

```
abc
CURSOR 1
```

Test Case 8

Input

```
5
TYPE a
TYPE b
UNDO
TYPE c
REDO
```

Output

```
ac
CURSOR 2
```

Test Case 9

Input

```
9
TYPE a
TYPE b
TYPE c
LEFT
BACKSPACE
UNDO
UNDO
REDO
REDO
```

Output

```
ac
CURSOR 1
```

Test Case 10

Input

```
8
BACKSPACE
LEFT
RIGHT
UNDO
REDO
TYPE z
UNDO
REDO
```

Output

```
z
CURSOR 1
```